Операционные системы

Лекция 5

Управление потоками

План лекции

- Различия процессов и потоков
- Устройство потоков
- Пользовательские потоки и потоки ядра
- Управление потоками в Windows
- Управление потоками в Linux (POSIX)

Процессы и потоки

До сих пор концепцию процесса мы характеризовали двумя свойствами:

- **Владение ресурсами** – как уже говорилось ранее, процесс включает в себя все необходимые ресурсы необходимые для выполнения программы
- **Планирование/управление выполнением** – процессы помогают в обеспечении мультизадачности путем чередования выполнения программы в рамках одного процесса с выполнением программ в сотнях других процессов, чем и занимается ОС выполняя планирование и диспетчеризацию

Процессы и потоки

Чтобы различать две приведенные выше характеристики, единицу диспетчеризации обычно называют **потоком** (**thread**) или **облегченным процессом** (**lightweight process**), а единицу владения ресурсами – **процессом** (**process**) или **заданием** (**task**)

Чтобы расставить вещи по своим местам стоит сказать, что существуют два понятия: **потоки управления** и **потоки выполнения**

И что самое главное – данные понятие вот вообще не одно и тоже, запомните!!! – Почему? Давайте проясним!

Процессы и потоки

Потоком управления считают логическую последовательность выполнения команд программы. Заметим что речь идёт в отношении именно **программы**!

Т.е. поток управления не более чем просто порядок, в котором выполняются команды и он просто определяет маршрут выполнения программы

Поток выполнения – это независимая последовательность инструкций в рамках ***процесса***, которая отвечает за реальное исполнение кода программы. Здесь же отметим, что речь идёт в отношении **процесса**!

Потоки выполнения (далее просто потоки) и будут целью данной лекции!

Процессы и потоки

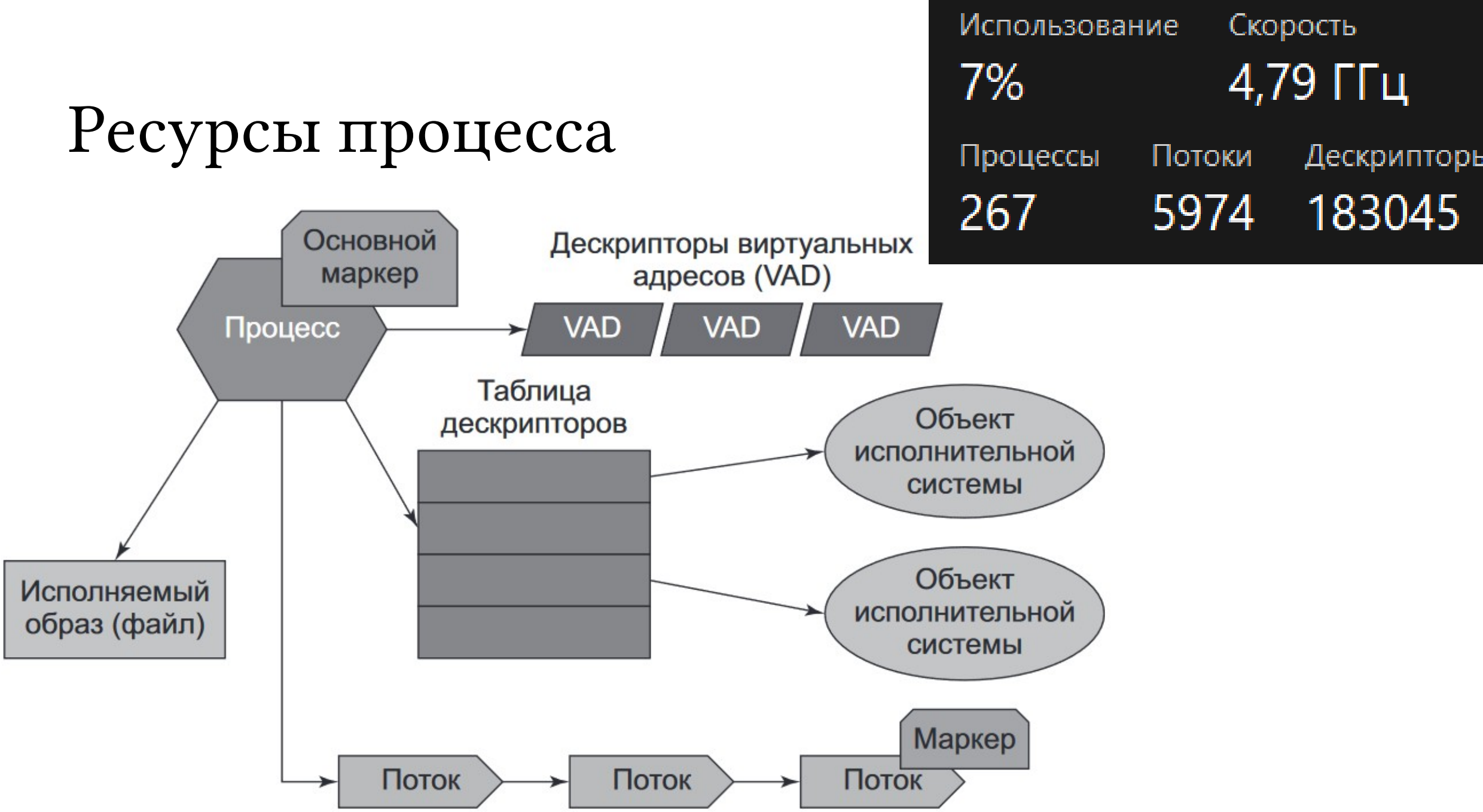
Собственно, что же такое **поток выполнения** с точки зрения ОС, а всё просто – это объект ядра операционной системы, которому выделяется процессорное время

Данный объект также является наименьшей единицей работы с точки зрения ядра ОС (если быть точнее, планировщика ОС)

И хотя данный объект выполняет какие-либо инструкции в рамках какого-либо процесса, сам поток и его процесс являются разными понятиями и должны рассматриваться по отдельности

Ещё раз! Процессы используются для **группировки ресурсов** в единое образование, а потоки являются «сущностью», **распределяемой для выполнения** на центральном процессоре

Ресурсы процесса



Процессы и потоки

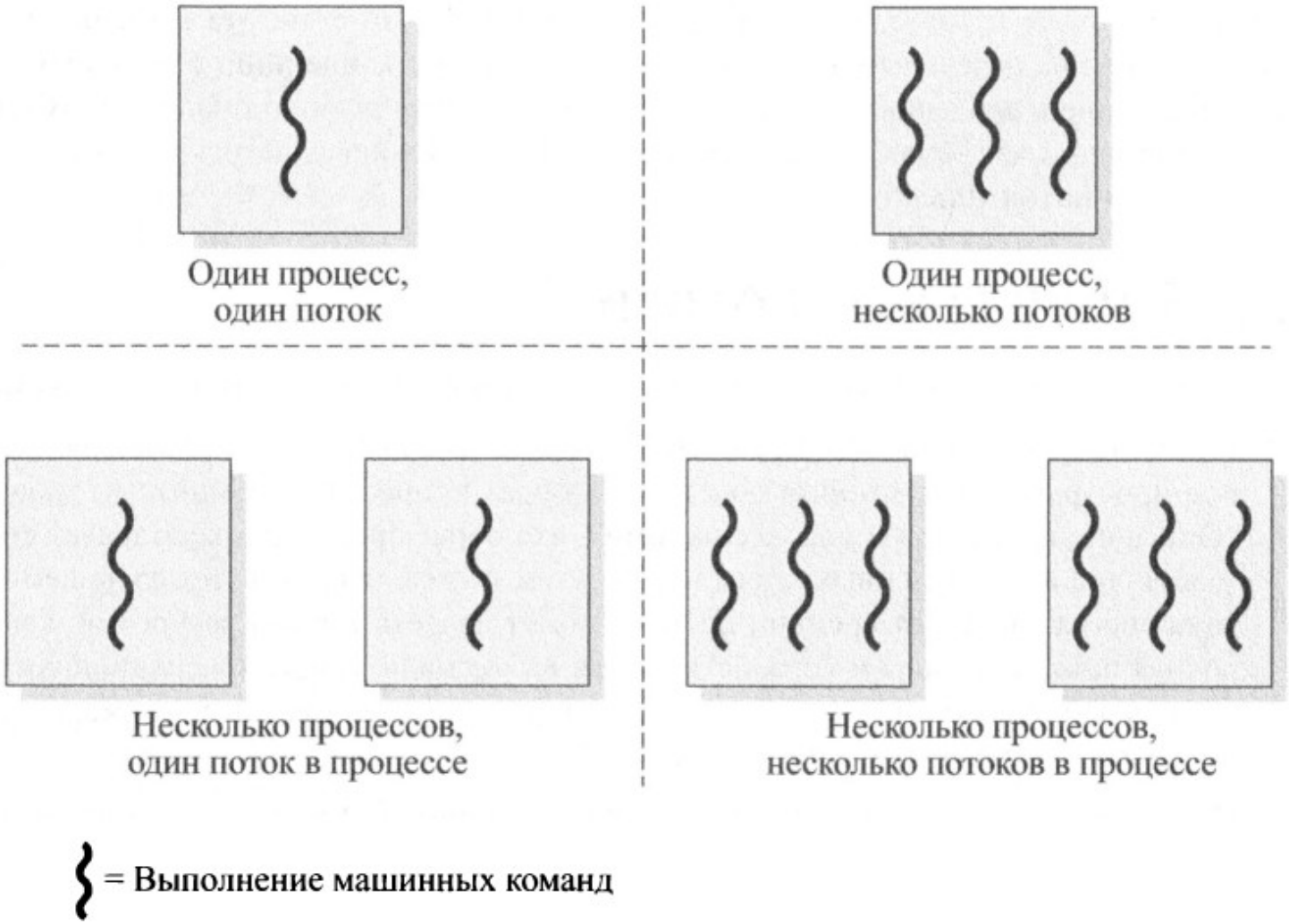
Потоки добавляют к модели процесса возможность реализации нескольких в значительной степени независимых друг от друга выполняемых задач в единой среде процесса

Наличие нескольких потоков, выполняемых «параллельно»* в рамках одного процесса, является аналогией наличия нескольких процессов, выполняемых «параллельно» на одном компьютере

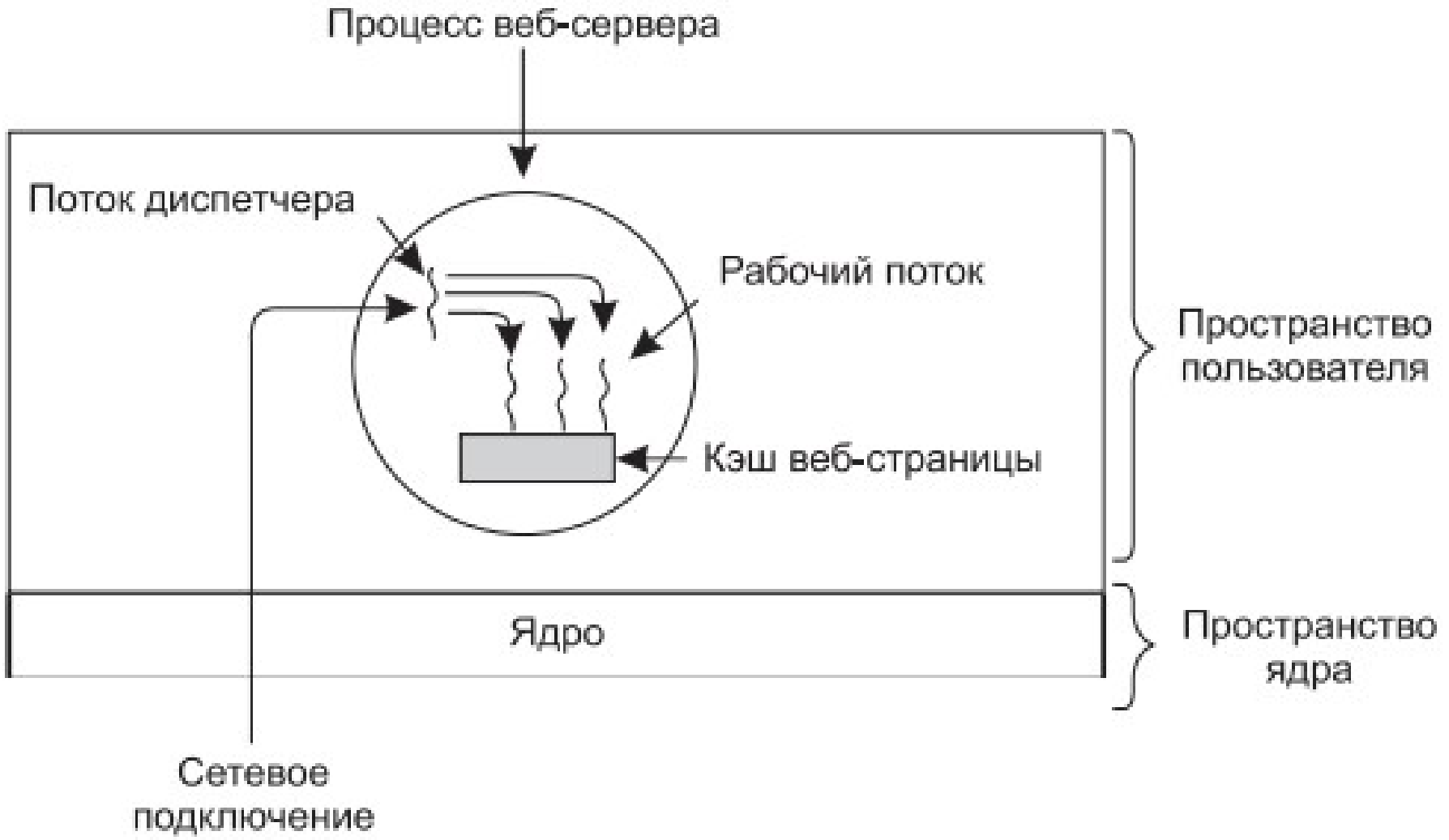
В первом случае потоки используют единое адресное пространство и другие ресурсы. А в последнем случае процессы используют общую физическую память, диски, принтеры и другие ресурсы

*Выше упомянутая параллельность весьма условна и не соблюдается во всех случаях

Процессы и потоки



Многопоточный веб-сервер



Процессы и потоки

Как уже говорилось одним из названий потоков является «облегчённый процесс» (LWP). Связано это с тем фактом, что потоки обладают некоторыми свойствами процессов

Термин же «**многопоточность**» обозначает возможность системы поддерживать работу множества потоков в рамках одного процесса

Если такой возможности нету, то система считается **однопоточной**

При этом, сами программы также классифицируются на **многопоточные** и **однопоточные** в зависимости от количества потоков выполнения которые они используют в своей работе

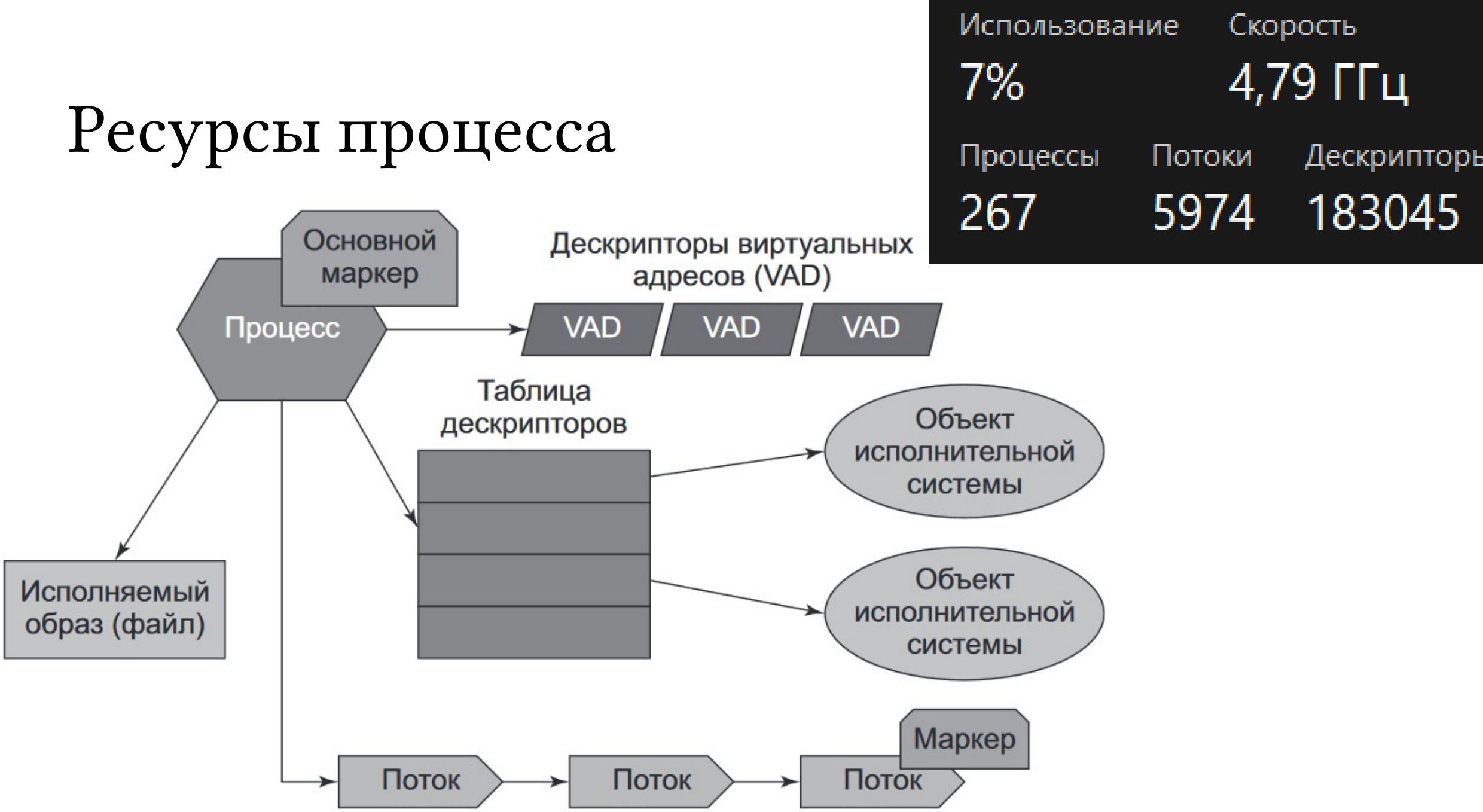
Процессы и потоки

И так как по определению любая последовательность ***выполнения*** инструкций называется потоком, именно поэтому в многопоточной среде процесс определяется как структурная единица распределения ресурсов, а также структурная единица защиты

С процессами связаны следующие элементы:

- Виртуальное адресное пространство, в котором содержится образ процесса
- Защищенный доступ к процессорам, другим процессам (при межпроцессном обмене информацией), файлам и ресурсам ввода-вывода (устройствам и каналам)

Ресурсы процесса



Процессы и потоки

В рамках процесса могут находиться один или несколько потоков, каждый из которых обладает следующими характеристиками:

- Состояние выполнения потока (выполняющийся, готовый к выполнению и т.д.)
- Сохраненный контекст не выполняющегося потока; один из способов рассмотрения потока – считать его независимым счетчиком команд, работающим в рамках процесса
- Стек выполнения
- Статическая память, выделяемая потоку для локальных переменных
- Доступ к памяти и ресурсам процесса, которому этот поток принадлежит; этот доступ разделяется всеми потоками данного процесса

Элементы, присущие каждому процессу	Элементы, присущие каждому потоку
Адресное пространство	Счетчик команд
Глобальные переменные	Регистры
Открытые файлы	Стек
Дочерние процессы	Состояние
Необработанные аварийные сигналы	
Сигналы и обработчики сигналов	
Учетная информация	

Процессы и потоки

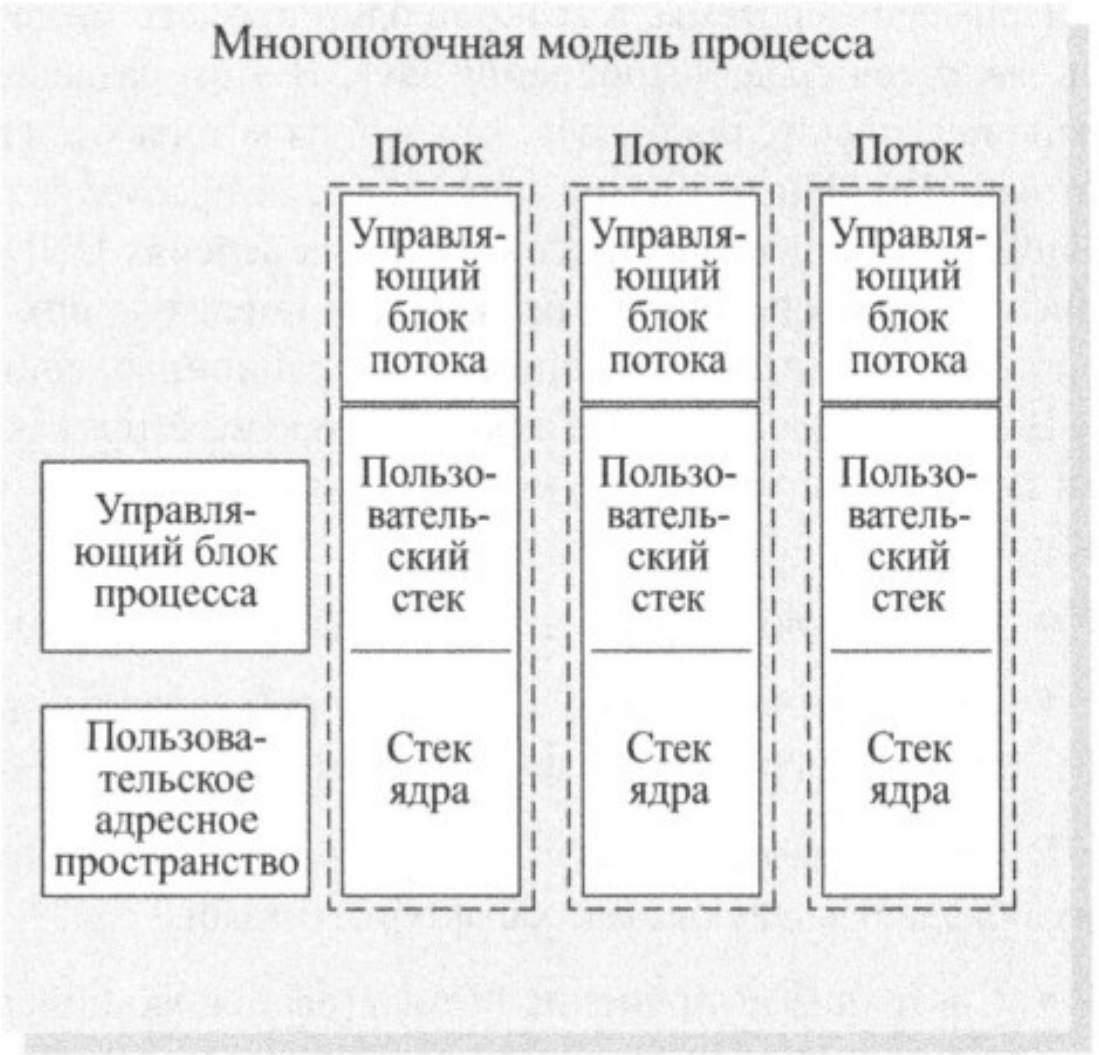
И хотя потоки имеют свой набор ресурсов, все потоки процесса разделяют между собой состояние и ресурсы процесса. Они находятся в одном и том же адресном пространстве и имеют доступ к одним и тем же данным

Если один поток изменяет в памяти какие-то данные, то другие потоки во время своего доступа к этим данным имеют возможность отследить эти изменения

Если один поток открывает файл с правом чтения, другие потоки данного процесса тоже могут читать из этого файла

Процессы и потоки





Процессы и потоки

Отвлечёмся на минуточку и подумаем, а что же такого в потоках? Ведь раньше системы прекрасно работали и без них

Основные преимущества использования потоков с точки зрения производительности таковы:

- Для создания нового потока в уже существующем процессе необходимо намного меньше времени, чем для создания нового процесса
- Поток можно завершить намного быстрее, чем процесс
- Переключение между потоками в рамках одного и того же процесса происходит намного быстрее переключения между процессами
- При использовании потоков повышается эффективность обмена информацией между двумя выполняющимися программами

Процессы и потоки

Однако как и у любого механизма ОС, у потоков есть и свои недостатки:

- При программировании приложения с множественными потоками необходимо постоянно думать о **потокобезопасности** (т. н. thread safety)
- Один неправильно работающий поток может повредить остальные, так как потоки делят общее адресное пространство
- Потоки конкурируют друг с другом в адресном пространстве. Стек и thread-local storage, занимая часть виртуального адресного пространства процесса, тем самым делают его недоступным для других потоков. Для встраиваемых (embedded) систем в условиях ограниченности ресурсов такое ограничение может иметь существенное значение

Процессы и потоки

Потокобезопасная функция – функция которая может быть вызвана одновременно из нескольких потоков без риска возникновения ошибок или непредсказуемого поведения

В общем случае функция называется **повторно входимой** или **реентерабельной** если она удовлетворяет следующим требованиям:

- не использует глобальные переменные, значения которых изменяются параллельно исполняемыми потоками
- не использует статические переменные, определенные внутри функции
- не возвращает указатель на статические данные, определенные внутри функции

Процессы и потоки

Когда лучше использовать поточную многозадачность:

- Ограниченность системных ресурсов
- Частое использование данных между подзадачами
- Частая синхронизация подзадач
- Поддержка языками и библиотеками
- Задачи с упором на вычисления (так называемые CPU-bound tasks)
- Написание GUI
- Задачи с требованием к масштабированию

Процессы и потоки

В операционной системе, поддерживающей потоки, планирование и диспетчеризация осуществляются на основе потоков; таким образом, большая часть информации о состоянии процесса, имеющей отношение к его выполнению, поддерживается в структурах данных на уровне потоков

Однако есть несколько действий, которые затрагивают все потоки процесса и которые операционная система должна поддерживать именно на этом уровне. Если процесс приостанавливается, то при этом предполагается, что его адресное пространство будет выгружено из основной памяти

Поскольку все потоки процесса используют одно и то же адресное пространство, все они должны одновременно перейти в состояние приостановленных. Соответственно, прекращение процесса приводит к прекращению всех составляющих его потоков

Процессы и потоки

Также стоит вспомнить, что при диспетчеризации происходит такое явление как **переключение контекста** (context switch) – в многозадачных ОС и средах – процесс прекращения выполнения процессором одной задачи (процесса, потока, нити) с сохранением всей необходимой информации и состояния, необходимых для последующего продолжения с прерванного места, и восстановления и загрузки состояния задачи, к выполнению которой переходит процессор

В случае с потоками, они также как и процессы имеют свой собственный **контекст**

Состояния потоков

Потоки, подобно процессам, характеризуются состояниями выполнения

Основными состояниями потоков, как и процессов, являются **состояние выполнения** потока, **состояние готовности** и **состояние блокировки**

Вообще говоря, **состояние приостановки** нет смысла связывать с потоками, потому что такие состояния логичнее рассматривать на уровне процессов

В частности, если процесс приостанавливается, обязательно приостанавливаются все его потоки, потому что все они совместно используют адресное пространство этого процесса

Состояния потоков

С изменением состояния потоков связаны такие четыре основных действия:

- **Порождение.** Обычно одновременно с новым процессом создается его поток. Далее, в рамках одного и того же процесса один поток может породить другой поток, определив его указатель команд и аргументы. Новый поток создается со своим контекстом регистров и стековым пространством, после чего он помещается в очередь готовых к выполнению потоков
- **Блокирование.** Если потоку нужно подождать наступления некоторого события, он блокируется (при этом сохраняются содержимое его пользовательских регистров, счетчика команд, а также указатели стеков). После этого процессор может перейти к выполнению другого готового потока

Состояния потоков

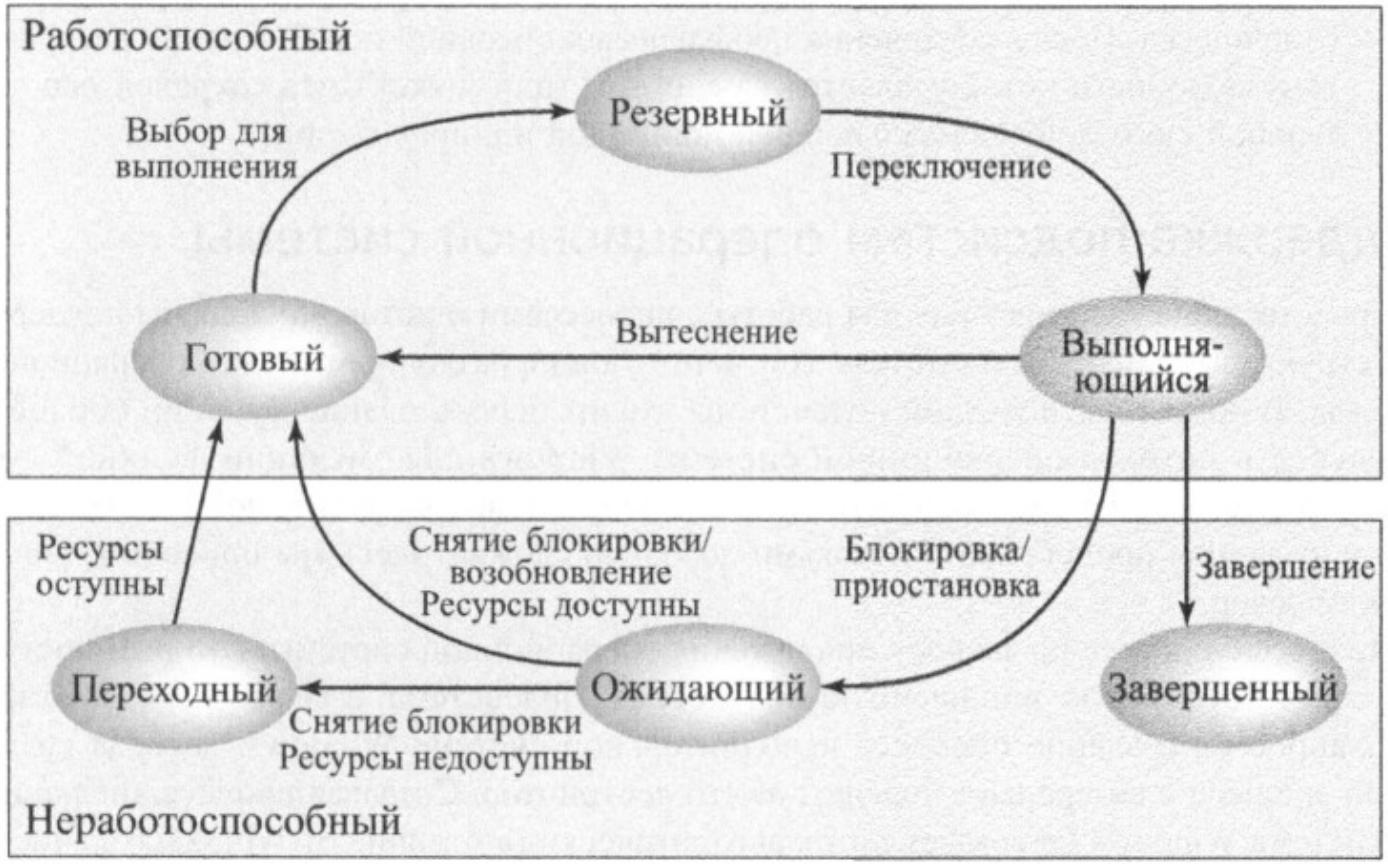
С изменением состояния потоков связаны такие четыре основных действия:

- **Разблокирование.** Когда наступает событие, ожидание которого блокировало поток, последний переходит в состояние готовности
- **Завершение.** После завершения потока его контекст регистров и стеки удаляются

Важно понимать что описанные состояния и действия являются базовыми и разработчики конкретных ОС могут их расширять по своему усмотрению

Состояния потоков Windows

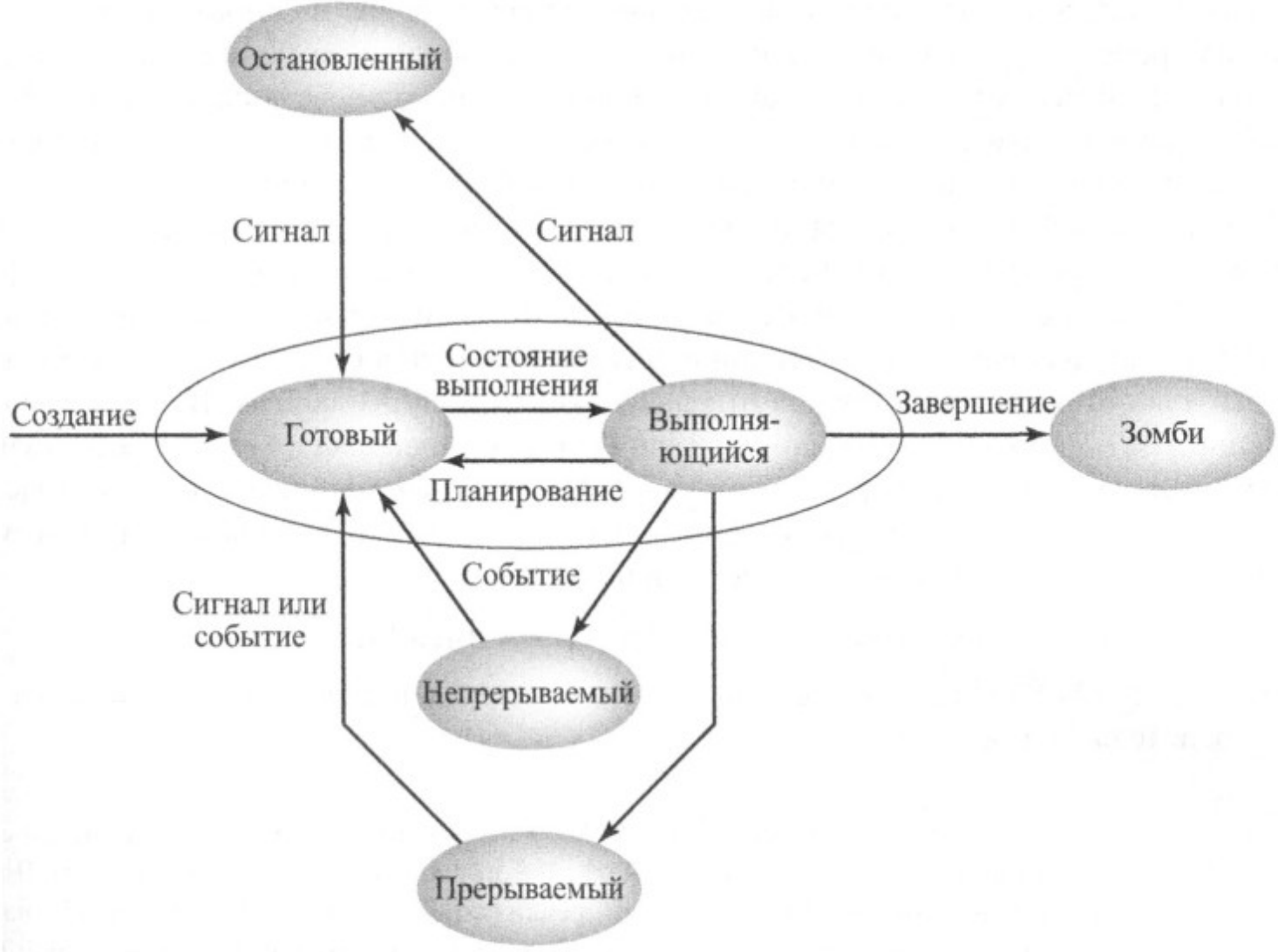
Так, например, в ОС Windows диаграмма состояний потоков выглядит следующим образом



Состояния потоков Linux

А в ОС Linux
следующим образом

Заметили кое-что?



Синхронизация потоков

Все потоки процесса используют одно и то же адресное пространство, как и другие ресурсы, например открытые файлы

Любое изменение какого-нибудь ресурса одним из потоков процесса оказывает влияние на другие потоки этого же процесса. Поэтому действия различных потоков необходимо **синхронизировать**, чтобы они не мешали друг другу или чтобы не повредили структуры данных

Например, если каждый из двух потоков будет пытаться добавить свой элемент в двунаправленный список, может быть потерян один из элементов (или нарушена целостность списка)

При рассмотрении синхронизации потоков возникают те же вопросы и используются те же методы, что и при синхронизации процессов (узнаем об этом в будущей лекции)

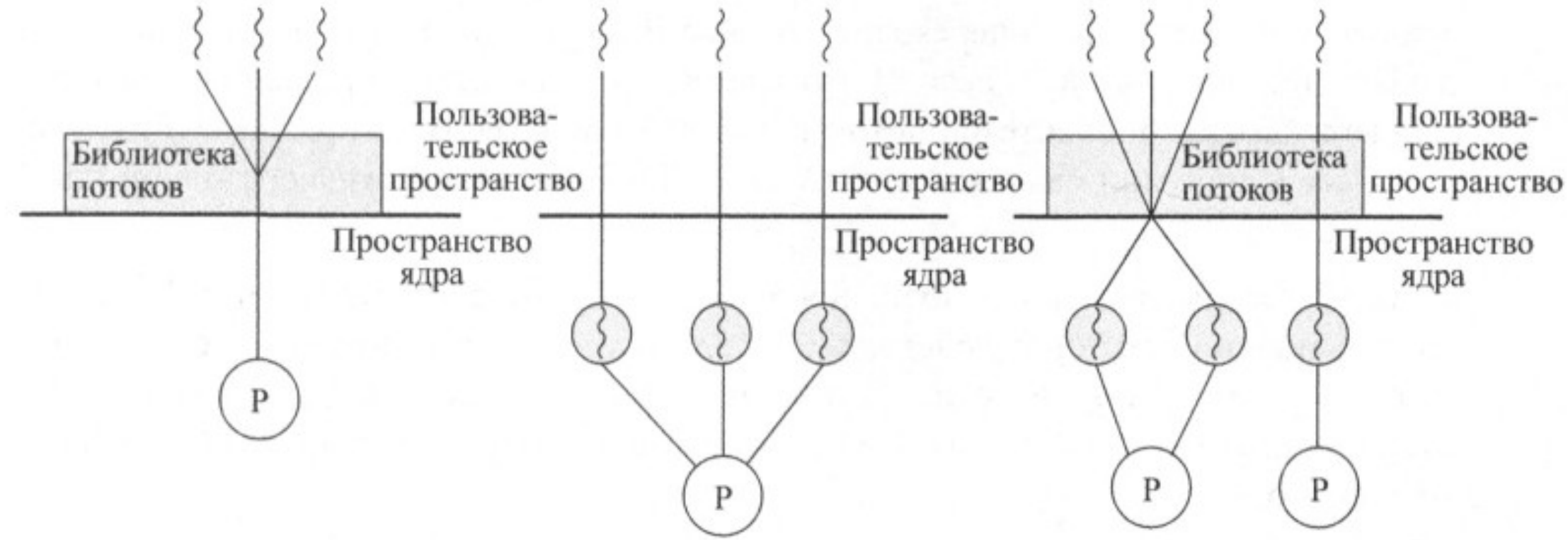
Типы потоков

Обычно выделяют две общие категории потоков:

- **Пользовательские потоки** или **потоки на уровне пользователя** (user-level threads – ULT)
- **Потоки на уровне ядра** (kemel-level threads – KLT)

Потоки второго типа в литературе иногда называются потоками, поддерживаемыми ядром или облегченными процессами (lightweight processes)

Типы потоков



а) Чисто пользовательский уровень б) Чисто уровень ядра в) Комбинация уровней

Пользовательские потоки

В программе, полностью состоящей из ULT-потоков, все действия по управлению потоками выполняются самим приложением; ядро, по сути, и не подозревает о существовании потоков

Чтобы приложение было многопоточным, его следует создавать с применением специальной библиотеки, представляющей собой пакет программ для работы с потоками на уровне ядра. Такая библиотека для работы с потоками содержит код, с помощью которого можно создавать и удалять потоки, производить обмен сообщениями и данными между потоками, планировать их выполнение, а также сохранять и восстанавливать их контекст

Пользовательские потоки

Все описанные события происходят в пользовательском пространстве в рамках одного процесса. Ядро не подозревает об этой деятельности. Оно продолжает осуществлять планирование процесса как единого целого и приписывать ему единое состояние выполнения (состояние готовности, состояние выполняющегося процесса, состояние блокировки и т. д.)

Пользовательские потоки

Применение потоков на пользовательском уровне обладает некоторыми преимуществами перед применением потоков на уровне ядра. К этим преимуществам относятся следующие:

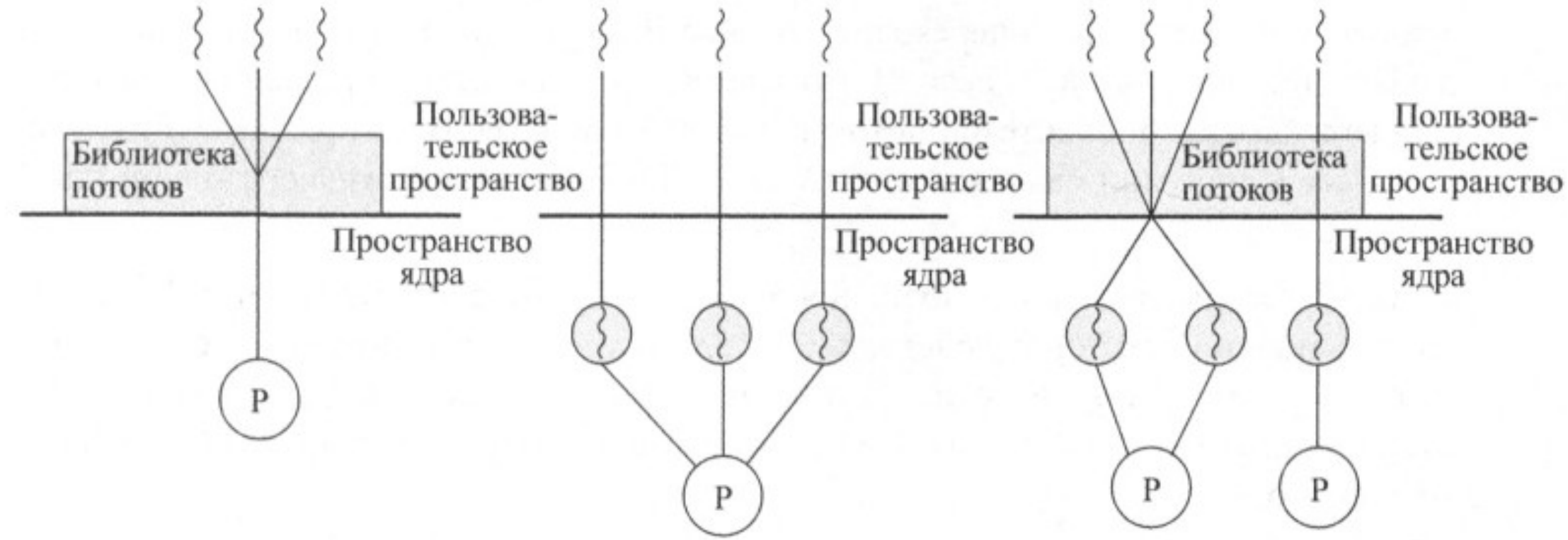
- Переключение потоков не предусматривает переход в режим ядра, а значит избегаем накладных расходов на переключение между режимами
- Планирование может выполняться с учетом специфики приложения. Алгоритм планирования может подбираться для конкретного приложения, причем это не повлияет на алгоритм планирования, заложенный в операционной системе
- Использование потоков на пользовательском уровне может работать в любой операционной системе. Для их поддержки в ядро системы не потребуется вносить никаких изменений

Пользовательские потоки

Применение потоков на пользовательском уровне обладает также двумя явными недостатками по сравнению с применением потоков на уровне ядра:

- В типичной операционной системе многие системные вызовы являются блокирующими. Когда в потоке, работающем на пользовательском уровне, выполняется системный вызов, блокируется не только данный поток, но и все потоки того процесса, к которому он относится
- В стратегии с наличием потоков только на пользовательском уровне приложение не может воспользоваться преимуществами многопроцессорной системы, так как ядро закрепляет за каждым процессом только один процессор. Поэтому несколько потоков одного и того же процесса не могут выполняться одновременно

Типы потоков



а) Чисто пользовательский уровень б) Чисто уровень ядра в) Комбинация уровней

Потоки уровня ядра

В программе, работа которой полностью основана на потоках, работающих на уровне ядра, все действия по управлению потоками выполняются ядром

В области приложений отсутствует код, предназначенный для управления потоками. Вместо него используется интерфейс прикладного программирования (application programming interface – API) средств ядра, управляющих потоками

Примером такого подхода является операционная система Windows

Потоки уровня ядра

Ядро поддерживает информацию контекста процесса как единого целого, а также контекстов каждого отдельного потока процесса Планирование выполняется ядром на основе потоков. С помощью такого подхода удается избавиться от двух упомянутых ранее основных недостатков потоков пользовательского уровня

Во-первых, ядро может одновременно осуществлять планирование работы нескольких потоков одного и того же процесса на нескольких процессорах.

Во-вторых, при блокировке одного из потоков процесса ядро может выбрать для выполнения другой поток этого же процесса

Еще одним преимуществом такого подхода является то, что сами процедуры ядра могут быть многопоточными

Потоки уровня ядра

Основным недостатком подхода с использованием потоков на уровне ядра по сравнению с использованием потоков на пользовательском уровне является то, что для передачи управления от одного потока другому в рамках одного и того же процесса приходится переключаться в режим ядра

Сравнение времени (в мс) выполнения операций на основе разных подходов:

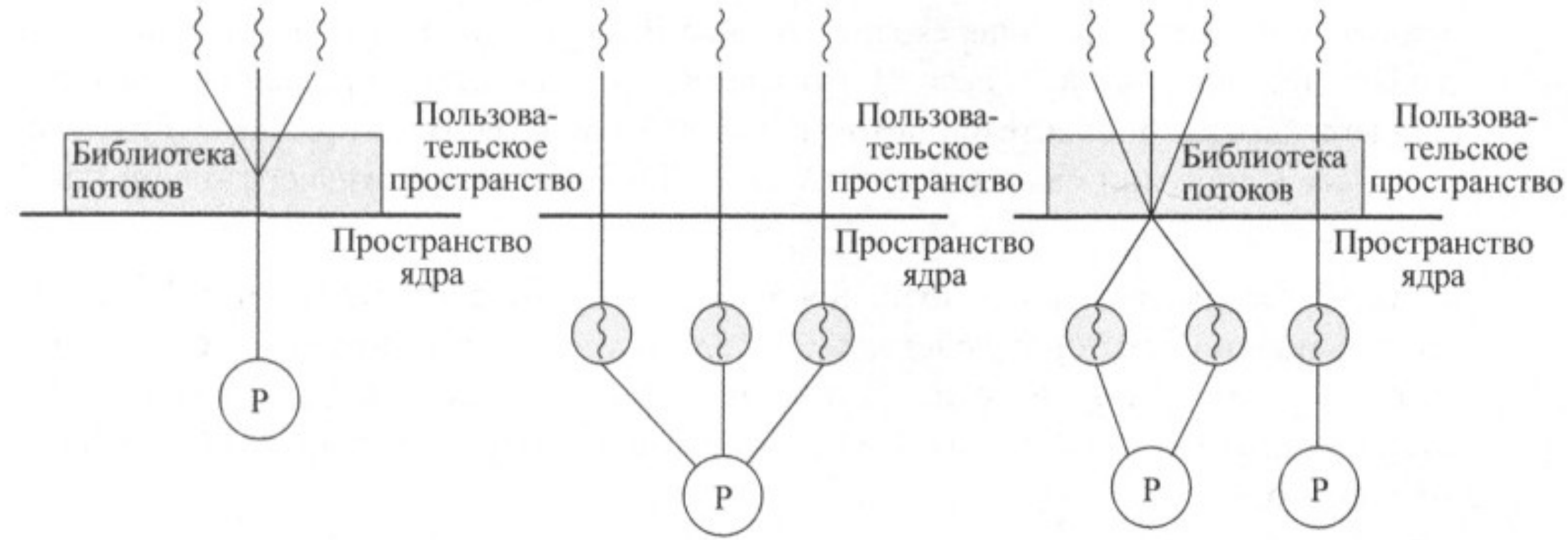
Операция	Пользовательские потоки	Потоки на уровне ядра	Процессы
Нулевое ветвление	34	948	11300
Ожидание сигнала	37	441	1840

Потоки уровня ядра

Таким образом, создается впечатление, что как применение многопоточности на уровне ядра дает выигрыш по сравнению с процессами, так и многопоточность на пользовательском уровне дает выигрыш по сравнению с многопоточностью на уровне ядра

Однако на деле возможность этого дополнительного выигрыша зависит от характера приложений. Если для большинства переключений потоков приложения необходим доступ к ядру, то схема с потоками на пользовательском уровне может работать не намного лучше, чем схема с потоками на уровне ядра

Типы потоков



а) Чисто пользовательский уровень б) Чисто уровень ядра в) Комбинация уровней

Комбинированный подход

В некоторых операционных системах применяется комбинирование потоков обоих видов. В комбинированных системах создание потоков выполняется полностью в пользовательском пространстве, там же, где и код планирования и синхронизации потоков в приложениях. Несколько потоков на пользовательском уровне, входящих в состав приложения, отображаются в такое же или меньшее число потоков на уровне ядра. Программист может изменять число потоков на уровне ядра, подбирая его таким, чтобы оно позволило достичь наилучших результатов

Комбинированный подход

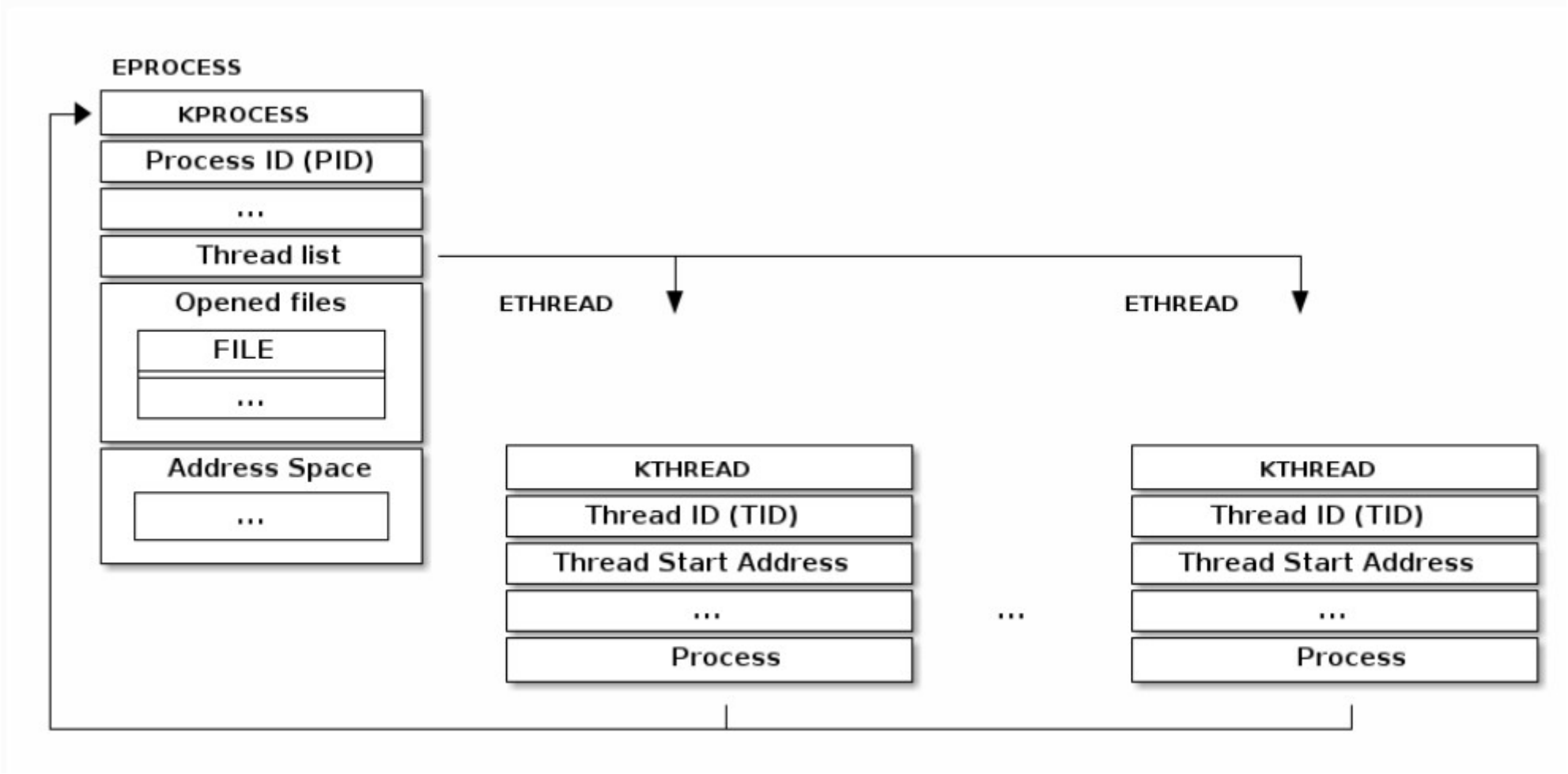
При комбинированном подходе несколько потоков одного и того же приложения могут выполняться одновременно на нескольких процессорах, а блокирующие системные вызовы не приводят к блокировке всего процесса. При надлежащей реализации такой подход будет сочетать в себе преимущества подходов, в которых применяются только потоки на пользовательском уровне или только потоки на уровне ядра, сводя недостатки каждого из этих подходов к минимуму

Потоки Windows и Linux

Также как и процессы, потоки являются в Windows объектами ядра и представлены структурой **ETHREAD**:

```
lkd> dt nt!_ETHREAD
   +0x000 Tcb              : _KTHREAD
   +0x430 CreateTime       : _LARGE_INTEGER
   +0x438 ExitTime         : _LARGE_INTEGER
   +0x438 KeyedWaitChain   : _LIST_ENTRY
   +0x448 PostBlockList    : _LIST_ENTRY
   +0x448 ForwardLinkShadow : Ptr64 Void
   +0x450 StartAddress     : Ptr64 Void
   +0x458 TerminationPort  : Ptr64 _TERMINATION_PORT
   +0x458 ReaperLink       : Ptr64 _ETHREAD
   +0x458 KeyedWaitValue   : Ptr64 Void
   +0x460 ActiveTimerListLock : Uint8B
   +0x468 ActiveTimerListHead : _LIST_ENTRY
   +0x478 Cid              : _CLIENT_ID
   +0x488 KeyedWaitSemaphore : _KSEMAPHORE
   +0x488 AlpcWaitSemaphore : _KSEMAPHORE
   +0x4a8 ClientSecurity   : _PS_CLIENT_SECURITY_CONTEXT
   +0x4b0 IrpList          : _LIST_ENTRY
   +0x4c0 TopLevelIrp      : Uint8B
   +0x4c8 DeviceToVerify   : Ptr64 _DEVICE_OBJECT
   +0x4d0 Win32StartAddress : Ptr64 Void
   +0x4d8 ChargeOnlySession : Ptr64 Void
```

Потоки Windows и Linux



Потоки Windows и Linux

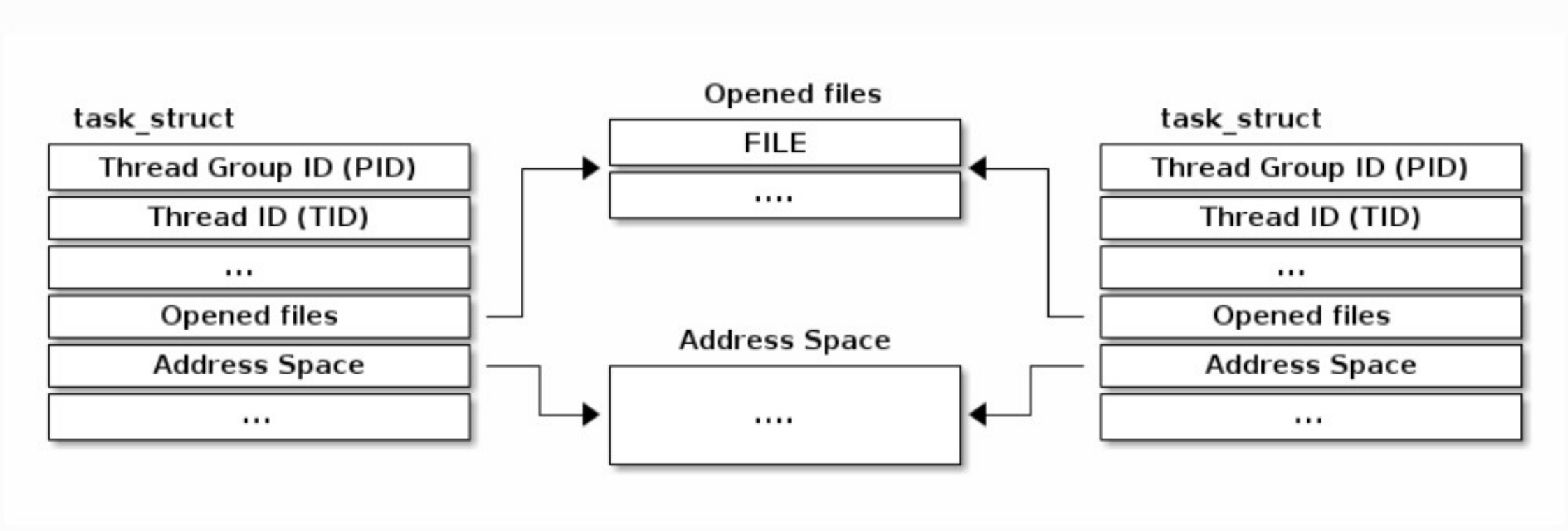
Если говорить о Linux, то тут возникает особая ситуация. Как вы могли заметить по диаграмме состояний – она идентична состояниям процесса

Всё дело в том, что Linux предоставляет уникальное решение, состоящее в том, что оно **не признает различие между потоками и процессами**

Пользовательские потоки отображаются на процессы уровня ядра. Несколько потоков пользовательского уровня, которые составляют единый процесс уровня пользователя, отображаются на процессы уровня ядра Linux, которые **разделяют один и тот же идентификатор группы**

Это позволяет данным процессам совместно использовать ресурсы, такие как файлы и память, и избегать необходимости переключения контекста, когда планировщик выполняет переключение между процессами в одной и той же группе

Потоки Windows и Linux



Потоки Windows и Linux

В Linux новый процесс создается путем копирования атрибутов текущего процесса

Можно клонировать новый процесс так, что он будет разделять такие ресурсы, как файлы, обработчики сигналов и виртуальная память. Когда два процесса разделяют одну и ту же виртуальную память, они функционируют как потоки в пределах одного процесса

Однако для потока не определяется отдельный тип структуры данных. Вместо обычного системного вызова **fork** процессы создаются в Linux с помощью вызова **clone**. Эта команда включает в себя в качестве аргументов набор флагов. Традиционный системный вызов **fork** реализуется в Linux как системный вызов **clone** со сброшенными флагами

Потоки Windows и Linux

Когда ядро Linux выполняет переключение контекста от одного процесса к другому, оно проверяет, является ли адрес каталога страниц текущего процесса тем же, что и у планируемого процесса

Если да, то они разделяют одно адресное пространство, так что переключение контекста в основном состоит просто в переходе из одного места в коде в другое

Хотя клонированные процессы, которые являются частью одной и той же группы процессов, могут разделить одно и то же пространство памяти, они не могут использовать одни и те же пользовательские стеки. Таким образом, вызов clone создает отдельный стек для каждого процесса

Потоки Windows и Linux

Создание потока (Windows) – **CreateThread**, **_beginthreadex**

Создание потока (POSIX) – **pthread_create**

```
uintptr_t _beginthreadex( // NATIVE CODE
   void *security,
   unsigned stack_size,
   unsigned ( __stdcall *start_address )( void * ),
   void *arglist,
   unsigned initflag,
   unsigned *thrdaddr
);
```

```
#include <pthread.h>

int pthread_create(pthread_t *restrict thread,
       const pthread_attr_t *restrict attr,
       void *(*start_routine)(void*), void *restrict arg);
```

Примечание: **_beginthreadex** не является частью WinAPI

Потоки Windows и Linux

Завершение потока (Windows) – **ExitThread**, **TerminateThread**, **_endthreadex**

Завершение потока (POSIX) – **pthread_exit**, **pthread_cancel**

```
void _endthread( void );
void _endthreadex(
   unsigned retval
);
```

```
void ExitThread(
  [in] DWORD dwExitCode
);
```

```
#include <pthread.h>

int pthread_cancel(pthread_t thread);
```

```
BOOL TerminateThread(
  [in, out] HANDLE hThread,
  [in]      DWORD  dwExitCode
);
```

```
#include <pthread.h>

_Noreturn void pthread_exit(void *value_ptr);
```

Потоки Windows и Linux

Завершение потока (Windows):

- функция потока возвращает управление (рекомендуемый способ); ***DWORD WINAPI ThreadFunc(LPVOID arg) {… return 0;}***
- поток самоуничтожается вызовом функции **ExitThread** (нежелательный способ);
- один из потоков данного или стороннего процесса вызывает функцию **TerminateThread** (нежелательный способ);
- завершается процесс, содержащий данный поток (тоже нежелательно) **ExitProcess(0);**

Потоки Windows и Linux

Завершение потока (POSIX):

- Начальная функция выполняет инструкцию **return**, указывая возвращаемое значение для потока
- Поток вызывает функцию **pthread_exit**
- Поток отменяется с помощью функции **pthread_cancel**
- Любой из потоков вызывает **exit** или главный поток выполняет инструкцию **return** (внутри функции main), что приводит к немедленному завершению всех потоков в процессе

Потоки Windows и Linux

Управление потоком (Windows) – **SuspendThread**, **ResumeThread**, **WaitForSingleObject**, **WaitForMultipleObjects**, **GetExitCodeThread**, **GetCurrentThread**

Управление потоком (POSIX) – **pthread_join**, **pthread_detach**, **pthread_self**, **pthread_once**

Полное описание функций **WinAPI** и **POSIX**

Thread Local Storage

Thread Local Storage – это механизм, который позволяет каждому потоку хранить свои собственные данные в отдельном пространстве, которое не доступно другим потокам. Это позволяет каждому потоку иметь свои собственные значения переменных, которые не будут перезаписаны другими потоками

Задействование данных, относящихся к отдельному потоку, позволяет сделать функцию потокобезопасной, не изменяя при этом ее интерфейс

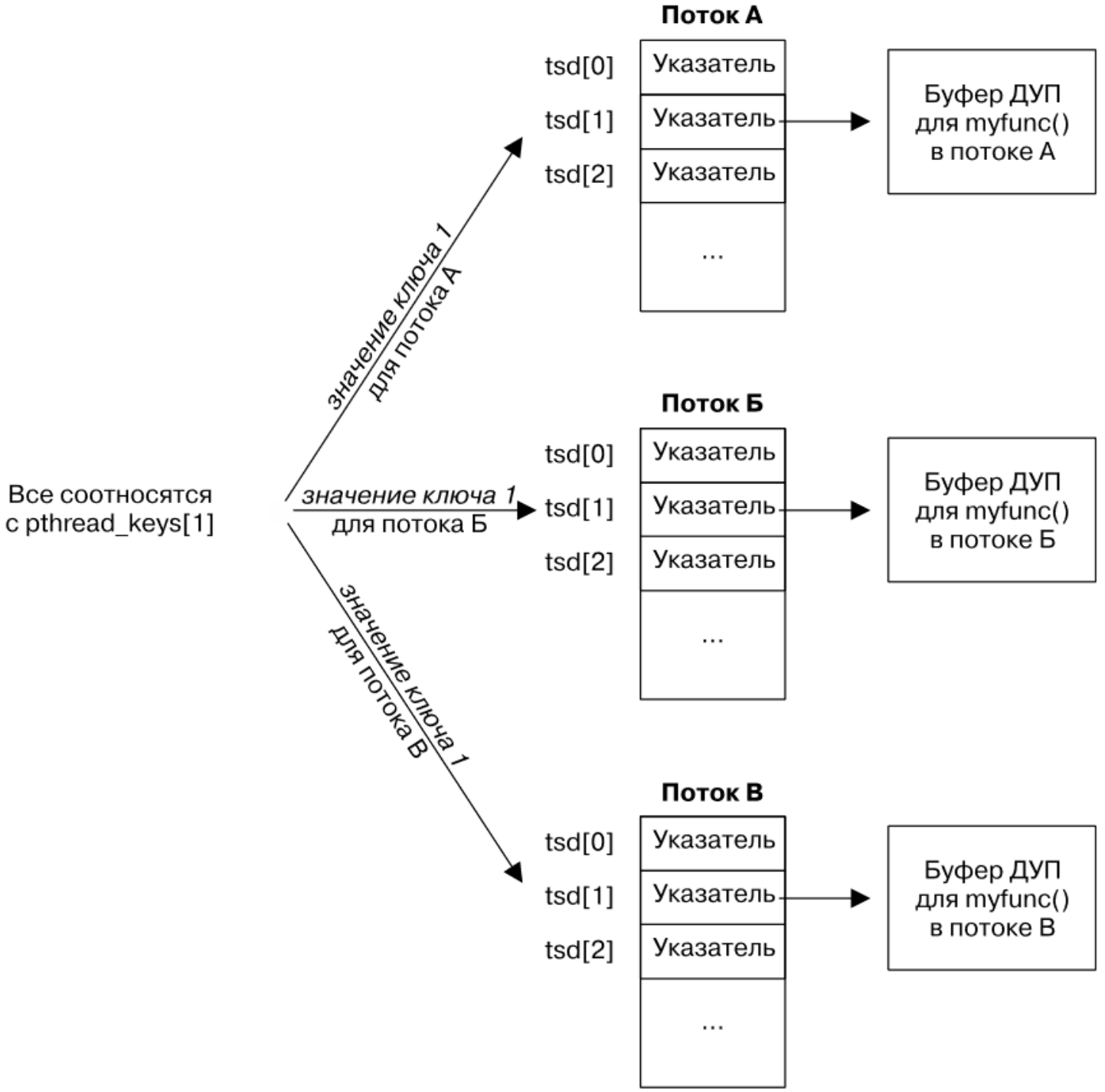
Thread Local Storage

Thread Local Storage API

Windows	POSIX
TlsAlloc	pthread_key_create
TlsSetValue, TlsGetValue	pthread_getspecific, pthread_setspecific
TlsFree	Второй параметр функции pthread_key_create

Thread Local Storage

Структура данных, которая используется для реализации указателей на данные уровня потока (ДУП)



Thread Local Storage

Существует также более простой достижения аналогичных результатов без использования специального API

Чтобы сделать глобальную или статическую переменную локальной для каждого потока, нужно просто указать при ее объявлении спецификатор

__thread (POSIX) и

__declspec(thread) (Windows)

Fibers

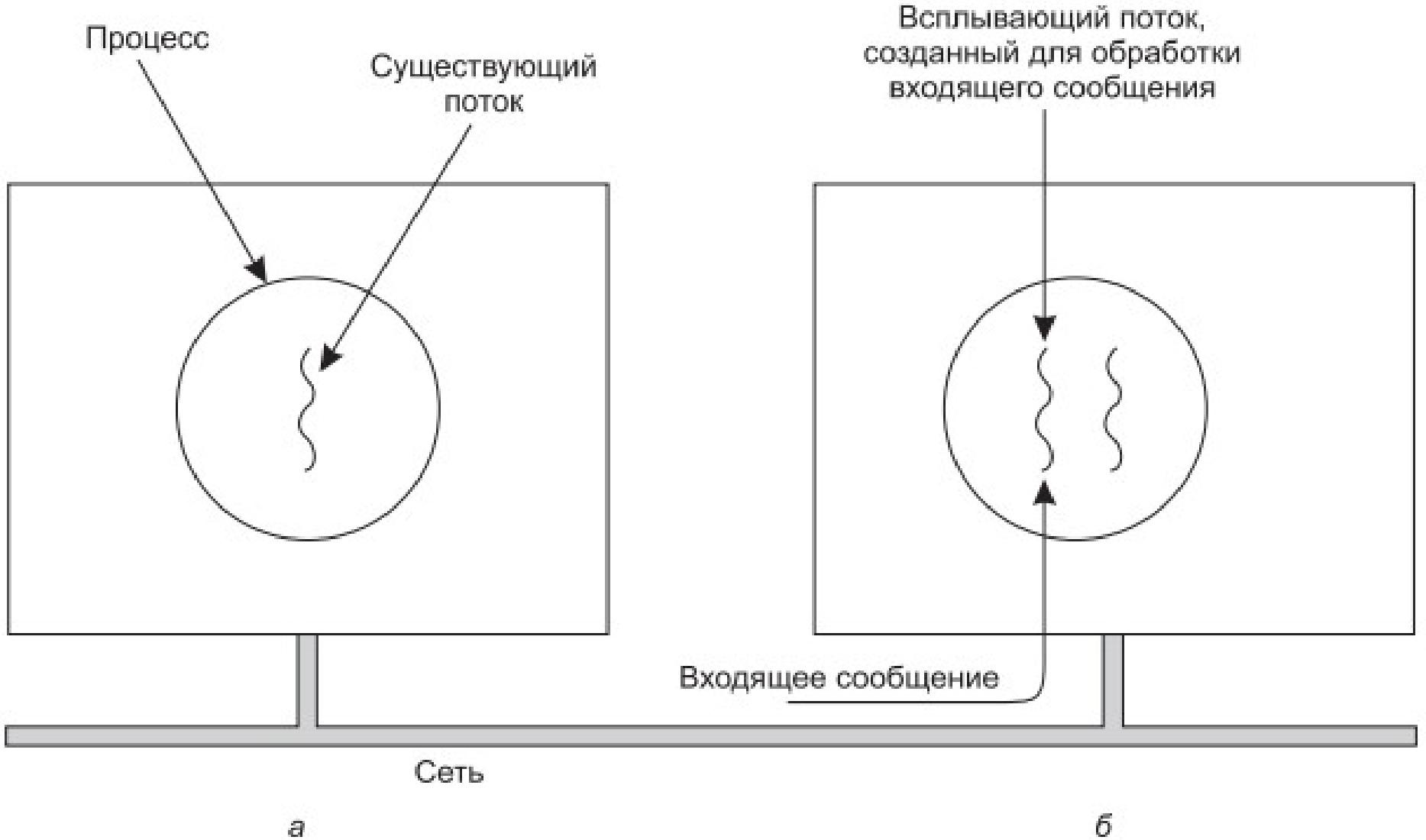
Стоит упомянуть, что в Windows существует такое понятие как **Волокна (Fibers)**

Fibers – это легковесные единицы выполнения с кооперативной многозадачностью, которые требуют ручного переключения контекста

Они предназначены для ещё большего контроля выполнения вашего пользовательского кода. Волокна создаются в рамках какого-либо потока и по сути вы берёте на себя обязательство по управлению ими в таком же смысле, как ОС управляет выполнением потоков

Волокна могут выполняться только пока выполняется владеющий ими поток

Всплывающие потоки



Операционные системы

Лекция 5

Управление потоками